

5. Project Development Phase

5.1 Introduction

The development phase converts the design into a functional application. It involves implementing the frontend, backend services, database, authentication, and Google Gemini text and image integration. The components are integrated through REST APIs to form a complete application.

5.2 Development Environment

Technology	Purpose
Python	Backend programming
FastAPI	REST API and backend framework
HTML / CSS	Web page structure and interface design
JavaScript	Client-side functionality and bubble editor
SQLite / PostgreSQL	Data storage
Google Gemini API	Script generation (text model) and panel artwork (image model)
Pillow / ReportLab	Image composition and PDF export
JWT	User authentication
Visual Studio Code	Development environment
Git / GitHub	Source code management

5.3 Application Development Architecture

User Interface → REST API → FastAPI Backend → Database & Storage, and Story Input → Prompt Builder → Gemini Text Model → Gemini Image Model → Image Processing → User Interface

This separates presentation, business logic, data management, and AI processing, making the application easier to maintain and extend.

5.4 Frontend Development

The frontend is built with HTML, CSS, and JavaScript. It includes login and registration pages, a story input form, a script editor, a character manager, a panel viewer with regenerate buttons, a speech bubble editor, a layout preview, and a gallery of saved comics. The interface is responsive, and progress indicators show which panel is currently being generated. Speech bubbles are drawn as draggable and resizable elements over the panel image so that text remains editable.

5.5 Backend Development

The backend is implemented in Python using FastAPI. It exposes REST endpoints for authentication, comics, characters, panel generation, bubble layouts, and export. Pydantic models validate request data, and dependency functions verify JWT tokens on protected routes. Image files are saved with unique names, and long-running image generation is handled per panel so that the interface can show results as they become available.

5.6 Gemini AI Integration

The backend communicates with the Google Gemini API through its Python SDK using two kinds of models. A Gemini text model receives a prompt template containing the story idea, genre, tone, art style, panel count, and character details, and is instructed to return a fixed JSON structure with the title, synopsis, characters, and panels. A Gemini image generation model then receives, for each panel, a prompt built from the scene description, camera angle, art style, and the stored character description blocks. Prompts instruct the model to create artwork without embedded text, so that dialogue is added later as editable speech bubbles. The backend parses the responses, handles errors, timeouts, and safety-blocked outputs, and returns clean results to the frontend.

5.7 Database Development

The database stores users, comics, characters, panels, and a generation log as described in the design phase. SQLAlchemy or direct SQL queries are used for data access, and each record is linked to the user through a user identifier so that users can access only their own data.

5.8 Authentication Implementation

Registration stores a hashed password. Login verifies the credentials and issues a JWT token, which the frontend sends with each request. Protected endpoints reject missing or invalid tokens.

5.9 Challenges and Solutions

Challenge	Solution
Characters look different in each panel	Stored character description blocks appended to every panel prompt; reference sheet; per-panel regeneration
Inconsistent script format from AI	Strict prompt instructions, JSON parsing with validation and fallback handling
Garbled text inside generated images	Images generated without text; dialogue overlaid as editable bubbles
Mixed art styles across panels	Same style keywords and settings applied to every panel prompt
API errors, delays, and blocked outputs	Try/except handling, retries, safety-block messages, and progress indicators
Keeping API key and user data secure	Environment variables, backend-only API calls, per-user data filtering